Week 4 Day 4 - Docker Compose Guide (< 3 hours)

🎯 Today's Goal

Replace multiple docker commands with a single configuration file and master container orchestration.

📚 Part 1: Understanding Docker Compose

The Problem with Manual Container Management

Day 3 approach (10+ commands):

bash

```
# Create network
docker network create myapp-network

# Create volumes
docker volume create postgres-data
docker volume create redis-data

# Start database (must wait for it to be ready)
docker run -d --name postgres --network myapp-network \
  -e POSTGRES_PASSWORD=secret \
  -v postgres-data:/var/lib/postgresql/data \
  postgres:15-alpine

# Wait manually
sleep 5

# Start cache
docker run -d --name redis --network myapp-network \
  -v redis-data:/data \
  redis:7-alpine

# Start API (hope database is ready!)
docker run -d --name api --network myapp-network \
  -p 8080:8080 \
  -e DB_HOST=postgres -e REDIS_HOST=redis \
  myapp:v1

# If anything fails, cleanup is a nightmare
docker stop api postgres redis
docker rm api postgres redis
docker network rm myapp-network
docker volume rm postgres-data redis-data
```

Problems:

- ❌ Too many commands to remember
- ❌ Manual startup order management
- ❌ No dependency tracking
- ❌ Hard to share setup with team
- ❌ Cleanup is tedious

The Docker Compose Solution

One file (docker-compose.yml):

```yaml
services:
  postgres:
    image: postgres:15-alpine
    volumes:
      - postgres-data:/var/lib/postgresql/data

  redis:
    image: redis:7-alpine

  api:
    build: ./api
    ports:
      - "8080:8080"
    depends_on:
      - postgres
      - redis

volumes:
  postgres-data:
```

One command:

```bash
docker-compose up -d
```

Benefits:

- ✅ All configuration in one file
- ✅ Automatic network creation
- ✅ Automatic volume management
- ✅ Dependency management
- ✅ Easy to version control
- ✅ Simple cleanup: `docker-compose down`

How Docker Compose Works

```
docker-compose.yml
      ↓
  Parse YAML
      ↓
1. Create network (automatic)
```

```
2. Create volumes (if needed)
3. Start services in order (depends_on)
4. Wait for health checks
5. Expose ports
      ↓
   All running!
```

🔧 Part 2: Your First docker-compose.yml

Basic Structure

yaml

```yaml
version: '3.8'              # Compose file version

services:                   # Define containers
  service-name:             # Container name
    image: image:tag        # OR
    build: ./path           # Build from Dockerfile

volumes:                    # Named volumes
  volume-name:

networks:                   # Custom networks
  network-name:
```

Simple Example: API + Database

yaml

```
version: '3.8'

services:
  # Database service
  postgres:
    image: postgres:15-alpine
    environment:
      POSTGRES_PASSWORD: secret
      POSTGRES_DB: myapp
    volumes:
      - db-data:/var/lib/postgresql/data

  # API service
  api:
    build: ./api
    ports:
      - "8080:8080"
    environment:
      DB_HOST: postgres
      DB_PASSWORD: secret
    depends_on:
      - postgres

volumes:
  db-data:
```

Start It Up

```bash
# Start all services
docker-compose up -d

# Check status
docker-compose ps

# View logs
docker-compose logs -f

# Stop everything
docker-compose down
```

What just happened?

1. Docker Compose created a network called `myapp_default`
2. Created volume `myapp_db-data`

3. Started postgres first (because of depends_on)
4. Started api after postgres
5. Both containers can talk via service names

🔗 Part 3: Service Dependencies

The depends_on Problem

```yaml
services:
  api:
    depends_on:
      - postgres  # Wait for container to START
```

Issue: Container "started" ≠ "ready to accept connections"

```
Time →
0s:  Postgres container starts
1s:  API starts (postgres container is running)
2s:  API tries to connect → FAILS! (postgres still initializing)
5s:  Postgres ready to accept connections
```

Solution: Health Checks

Add health check to postgres:

```yaml
services:
  postgres:
    image: postgres:15-alpine
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 10s    # Check every 10 seconds
      timeout: 5s      # Timeout after 5 seconds
      retries: 5       # Try 5 times before giving up
```

Use condition in depends_on:

```yaml
```

```yaml
services:
  api:
    depends_on:
      postgres:
        condition: service_healthy  # Wait for HEALTHY status
```

Now the flow is:

```
0s:  Postgres container starts
0-30s: Health checks running (every 10s)
30s: Postgres reports HEALTHY
30s: API starts (postgres is actually ready!)
31s: API connects successfully ✅
```

Understanding Health Check Results

bash

```
# Check service health status
docker-compose ps

# Output:
# NAME        STATUS                  PORTS
# postgres    Up (healthy)            5432/tcp
# api         Up                      0.0.0.0:8080->8080/tcp
```

Status meanings:

- `starting` - Container just started, health checks beginning
- `healthy` - Passed health check
- `unhealthy` - Failed health check after retries
- `Up` - No health check defined

💊 Part 4: Health Checks Deep Dive

Types of Health Checks

1. Command-based (PostgreSQL example):

yaml

```yaml
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U postgres"]
  interval: 10s
  timeout: 5s
  retries: 5
```

2. HTTP-based (API example):

```yaml
healthcheck:
  test: ["CMD", "curl", "-f", "http://localhost:8080/health"]
  interval: 30s
  timeout: 3s
  retries: 3
  start_period: 5s  # Grace period before starting checks
```

3. Custom script:

```yaml
healthcheck:
  test: ["CMD", "/app/healthcheck.sh"]
  interval: 30s
```

Why start_period Matters

```yaml
healthcheck:
  test: ["CMD", "curl", "-f", "http://localhost:8080/health"]
  interval: 10s
  start_period: 40s  # Don't fail checks for first 40s
```

Without start_period:

```
0s:  Container starts
10s: Health check #1 → FAIL (app still loading)
20s: Health check #2 → FAIL
30s: Health check #3 → FAIL
Container marked unhealthy! ❌
```

With start_period:

```
0s:  Container starts
10s: Health check (ignored - within start_period)
20s: Health check (ignored)
30s: Health check (ignored)
40s: Health check #1 → SUCCESS ✅
Container marked healthy!
```

Implementing Health Endpoint in Your API

```go
func healthHandler(w http.ResponseWriter, r *http.Request) {
    health := map[string]string{
        "status": "healthy",
    }

    // Check database
    if err := db.Ping(); err != nil {
        health["database"] = "unhealthy"
        health["status"] = "degraded"
        w.WriteHeader(http.StatusServiceUnavailable)
    } else {
        health["database"] = "healthy"
    }

    // Check Redis
    if _, err := redisClient.Ping(ctx).Result(); err != nil {
        health["redis"] = "unhealthy"
        health["status"] = "degraded"
        w.WriteHeader(http.StatusServiceUnavailable)
    } else {
        health["redis"] = "healthy"
    }

    json.NewEncoder(w).Encode(health)
}
```

Response when healthy:

```json
{
  "status": "healthy",
  "database": "healthy",
  "redis": "healthy"
}
```

Response when degraded:

```json
{
  "status": "degraded",
  "database": "unhealthy",
  "redis": "healthy"
}
```

🌍 Part 5: Environment Variables

Three Ways to Set Variables

1. Directly in compose file:

```yaml
services:
  api:
    environment:
      DB_HOST: postgres
      DB_PASSWORD: secret
      DEBUG: "true"
```

2. Using .env file (recommended):

Create .env:

```bash
DB_PASSWORD=mysecretpassword
API_PORT=8080
REDIS_PASSWORD=redissecret
```

Reference in docker-compose.yml:

```yaml
services:
  api:
    ports:
      - "${API_PORT}:8080"
    environment:
      DB_PASSWORD: ${DB_PASSWORD}
      REDIS_PASSWORD: ${REDIS_PASSWORD}
```

3. Environment file for service:

Create api.env:

```bash
LOG_LEVEL=debug
MAX_CONNECTIONS=100
TIMEOUT=30s
```

Use in compose:

```yaml
services:
  api:
    env_file:
      - ./api.env
```

Variable Substitution and Defaults

```yaml
services:
  api:
    ports:
      - "${API_PORT:-8080}:8080"  # Default to 8080 if not set
    environment:
      DB_HOST: ${DB_HOST:-postgres}
      LOG_LEVEL: ${LOG_LEVEL:-info}
```

Syntax: `${VARIABLE:-default_value}`

🎨 Part 6: Development vs Production

Base Configuration (docker-compose.yml)

```yaml
```

```yaml
version: '3.8'

services:
  postgres:
    image: postgres:15-alpine
    environment:
      POSTGRES_DB: myapp
    volumes:
      - postgres-data:/var/lib/postgresql/data

  api:
    build: ./api
    environment:
      DB_HOST: postgres

volumes:
  postgres-data:
```

Development Overrides (docker-compose.dev.yml)

yaml

```yaml
version: '3.8'

services:
  api:
    build:
      context: ./api
      target: builder  # Stop at builder stage
    volumes:
      - ./api:/app  # Mount source code for hot reload
    environment:
      DEBUG: "true"
      LOG_LEVEL: debug
    command: go run main.go  # Use go run instead of compiled binary

  postgres:
    ports:
      - "5432:5432"  # Expose for database GUI tools
```

Production Overrides (docker-compose.prod.yml)

yaml

```yaml
version: '3.8'

services:
  api:
    restart: always
    deploy:
      replicas: 3
      resources:
        limits:
          cpus: '0.5'
          memory: 512M
    logging:
      driver: "json-file"
      options:
        max-size: "10m"
        max-file: "3"
```

Using Different Configurations

```bash
# Development
docker-compose -f docker-compose.yml -f docker-compose.dev.yml up

# Production
docker-compose -f docker-compose.yml -f docker-compose.prod.yml up -d

# Or use Makefile
make dev   # Runs dev config
make prod  # Runs prod config
```

🎯 Part 7: Practice Exercise - E-commerce Backend (60 mins)

Goal

Build a multi-service e-commerce backend with:

- **API** - Product catalog and orders
- **PostgreSQL** - Product and order data
- **Redis** - Session cache and product catalog cache
- **Worker** - Process orders asynchronously

Step 1: Project Structure

bash

```
mkdir ecommerce && cd ecommerce
mkdir api worker
```

Step 2: API Service (api/main.go)

go

```go
package main

import (
    "context"
    "database/sql"
    "encoding/json"
    "fmt"
    "log"
    "net/http"
    "os"
    "time"

    _ "github.com/lib/pq"
    "github.com/redis/go-redis/v9"
)

var (
    db          *sql.DB
    redisClient *redis.Client
    ctx         = context.Background()
)

type Product struct {
    ID    int     `json:"id"`
    Name  string  `json:"name"`
    Price float64 `json:"price"`
    Stock int     `json:"stock"`
}

type Order struct {
    ID        int       `json:"id"`
    ProductID int       `json:"product_id"`
    Quantity  int       `json:"quantity"`
    Status    string    `json:"status"`
    CreatedAt time.Time `json:"created_at"`
}

func initDB() error {
    connStr := fmt.Sprintf(
        "host=%s user=%s password=%s dbname=%s sslmode=disable",
        getEnv("DB_HOST", "localhost"),
        getEnv("DB_USER", "postgres"),
        getEnv("DB_PASSWORD", "secret"),
        getEnv("DB_NAME", "ecommerce"),
    )
```

```
    var err error
    db, err = sql.Open("postgres", connStr)
    if err != nil {
        return err
    }

    // Wait for database
    for i := 0; i < 30; i++ {
        if err = db.Ping(); err == nil {
            break
        }
        time.Sleep(time.Second)
    }
    if err != nil {
        return err
    }

    // Create tables
    schema := `
    CREATE TABLE IF NOT EXISTS products (
        id SERIAL PRIMARY KEY,
        name VARCHAR(200) NOT NULL,
        price DECIMAL(10,2) NOT NULL,
        stock INT NOT NULL DEFAULT 0
    );

    CREATE TABLE IF NOT EXISTS orders (
        id SERIAL PRIMARY KEY,
        product_id INT REFERENCES products(id),
        quantity INT NOT NULL,
        status VARCHAR(20) DEFAULT 'pending',
        created_at TIMESTAMP DEFAULT NOW()
    );

    INSERT INTO products (name, price, stock) VALUES
        ('Laptop', 999.99, 10),
        ('Mouse', 29.99, 50),
        ('Keyboard', 79.99, 30)
    ON CONFLICT DO NOTHING;
    `
    _, err = db.Exec(schema)
    return err
}

func initRedis() error {
    redisClient = redis.NewClient(&redis.Options{
        Addr: fmt.Sprintf("%s:6379", getEnv("REDIS_HOST", "localhost")),
```

```
    })
    return redisClient.Ping(ctx).Err()
}

func getEnv(key, def string) string {
    if val := os.Getenv(key); val != "" {
        return val
    }
    return def
}

func healthHandler(w http.ResponseWriter, r *http.Request) {
    health := map[string]string{"status": "healthy"}
    if err := db.Ping(); err != nil {
        health["database"] = "unhealthy"
        health["status"] = "degraded"
    } else {
        health["database"] = "healthy"
    }
    if _, err := redisClient.Ping(ctx).Result(); err != nil {
        health["redis"] = "unhealthy"
        health["status"] = "degraded"
    } else {
        health["redis"] = "healthy"
    }
    w.Header().Set("Content-Type", "application/json")
    json.NewEncoder(w).Encode(health)
}

func getProductsHandler(w http.ResponseWriter, r *http.Request) {
    // Try cache
    cached, err := redisClient.Get(ctx, "products").Result()
    if err == nil {
        w.Header().Set("Content-Type", "application/json")
        w.Header().Set("X-Cache", "HIT")
        w.Write([]byte(cached))
        return
    }

    rows, err := db.Query("SELECT id, name, price, stock FROM products")
    if err != nil {
        http.Error(w, err.Error(), 500)
        return
    }
    defer rows.Close()

    products := []Product{}
```

```
    for rows.Next() {
        var p Product
        rows.Scan(&p.ID, &p.Name, &p.Price, &p.Stock)
        products = append(products, p)
    }

    data, _ := json.Marshal(products)
    redisClient.Set(ctx, "products", data, 60*time.Second)

    w.Header().Set("Content-Type", "application/json")
    w.Header().Set("X-Cache", "MISS")
    w.Write(data)
}

func createOrderHandler(w http.ResponseWriter, r *http.Request) {
    var order Order
    json.NewDecoder(r.Body).Decode(&order)

    err := db.QueryRow(
        "INSERT INTO orders (product_id, quantity) VALUES ($1, $2) RETURNING id, created_at",
        order.ProductID, order.Quantity,
    ).Scan(&order.ID, &order.CreatedAt)

    if err != nil {
        http.Error(w, err.Error(), 500)
        return
    }

    order.Status = "pending"

    // Publish to Redis for worker
    orderData, _ := json.Marshal(order)
    redisClient.Publish(ctx, "orders", orderData)

    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(http.StatusCreated)
    json.NewEncoder(w).Encode(order)
}

func main() {
    if err := initDB(); err != nil {
        log.Fatal("DB init failed:", err)
    }
    if err := initRedis(); err != nil {
        log.Fatal("Redis init failed:", err)
    }
```

```
    http.HandleFunc("/health", healthHandler)
    http.HandleFunc("/products", getProductsHandler)
    http.HandleFunc("/orders", createOrderHandler)

    log.Println("API starting on :8080")
    log.Fatal(http.ListenAndServe(":8080", nil))
}
```

Step 3: Worker Service (worker/main.go)

```
go
```

```go
package main

import (
    "context"
    "database/sql"
    "encoding/json"
    "fmt"
    "log"
    "os"
    "time"

    _ "github.com/lib/pq"
    "github.com/redis/go-redis/v9"
)

var (
    db          *sql.DB
    redisClient *redis.Client
    ctx         = context.Background()
)

type Order struct {
    ID        int    `json:"id"`
    ProductID int    `json:"product_id"`
    Quantity  int    `json:"quantity"`
    Status    string `json:"status"`
}

func initDB() error {
    connStr := fmt.Sprintf(
        "host=%s user=%s password=%s dbname=%s sslmode=disable",
        os.Getenv("DB_HOST"),
        os.Getenv("DB_USER"),
        os.Getenv("DB_PASSWORD"),
        os.Getenv("DB_NAME"),
    )

    var err error
    db, err = sql.Open("postgres", connStr)
    if err != nil {
        return err
    }

    for i := 0; i < 30; i++ {
        if err = db.Ping(); err == nil {
            break
```

```
        }
        time.Sleep(time.Second)
    }
    return err
}

func initRedis() error {
    redisClient = redis.NewClient(&redis.Options{
        Addr: fmt.Sprintf("%s:6379", os.Getenv("REDIS_HOST")),
    })
    return redisClient.Ping(ctx).Err()
}

func processOrder(order Order) error {
    log.Printf("Processing order %d", order.ID)

    // Simulate order processing
    time.Sleep(2 * time.Second)

    // Update order status
    _, err := db.Exec(
        "UPDATE orders SET status = $1 WHERE id = $2",
        "completed", order.ID,
    )

    if err == nil {
        log.Printf("Order %d completed", order.ID)
    }
    return err
}

func main() {
    if err := initDB(); err != nil {
        log.Fatal("DB init failed:", err)
    }
    if err := initRedis(); err != nil {
        log.Fatal("Redis init failed:", err)
    }

    // Subscribe to orders channel
    pubsub := redisClient.Subscribe(ctx, "orders")
    defer pubsub.Close()

    log.Println("Worker started, waiting for orders...")

    for msg := range pubsub.Channel() {
        var order Order
```

```
        if err := json.Unmarshal([]byte(msg.Payload), &order); err != nil {
            log.Printf("Error parsing order: %v", err)
            continue
        }

        if err := processOrder(order); err != nil {
            log.Printf("Error processing order: %v", err)
        }
    }
}
```

Step 4: Docker Compose (docker-compose.yml)

yaml

```yaml
version: '3.8'

services:
  postgres:
    image: postgres:15-alpine
    container_name: ecommerce-db
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: ${DB_PASSWORD:-secret}
      POSTGRES_DB: ecommerce
    volumes:
      - postgres-data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 10s
      timeout: 5s
      retries: 5
    restart: unless-stopped

  redis:
    image: redis:7-alpine
    container_name: ecommerce-cache
    command: redis-server --appendonly yes
    volumes:
      - redis-data:/data
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 10s
      timeout: 3s
      retries: 5
    restart: unless-stopped

  api:
    build:
      context: ./api
      dockerfile: Dockerfile
    container_name: ecommerce-api
    ports:
      - "${API_PORT:-8080}:8080"
    environment:
      DB_HOST: postgres
      DB_USER: postgres
      DB_PASSWORD: ${DB_PASSWORD:-secret}
      DB_NAME: ecommerce
      REDIS_HOST: redis
    depends_on:
```

```
    postgres:
      condition: service_healthy
    redis:
      condition: service_healthy
  restart: unless-stopped

worker:
  build:
    context: ./worker
    dockerfile: Dockerfile
  container_name: ecommerce-worker
  environment:
    DB_HOST: postgres
    DB_USER: postgres
    DB_PASSWORD: ${DB_PASSWORD:-secret}
    DB_NAME: ecommerce
    REDIS_HOST: redis
  depends_on:
    postgres:
      condition: service_healthy
    redis:
      condition: service_healthy
  restart: unless-stopped

volumes:
  postgres-data:
  redis-data:
```

Step 5: Test the System

bash

```
# Start everything
docker-compose up -d

# Wait for services to be healthy
docker-compose ps

# View products (cached)
curl http://localhost:8080/products

# Create an order
curl -X POST http://localhost:8080/orders \
  -H "Content-Type: application/json" \
  -d '{"product_id":1,"quantity":2}'

# Watch worker process the order
docker-compose logs -f worker

# Check order was processed
docker-compose exec postgres psql -U postgres -d ecommerce \
  -c "SELECT * FROM orders;"
```

🐛 Common Issues & Solutions

Issue 1: "Service 'X' is unhealthy"

Check health status:

bash

```
docker-compose ps
docker-compose logs <service-name>
```

Common causes:

- Service taking longer than start_period
- Wrong health check command
- Dependency not actually ready

Solution:

yaml

```
healthcheck:
  start_period: 60s  # Increase grace period
  interval: 10s
  retries: 10        # More retries
```

Issue 2: Services can't communicate

Symptom: "no such host" or "connection refused"

Debug:

```bash
# Check if services are on same network
docker-compose exec api ping postgres

# Check network
docker network inspect <project>_default
```

Solution: Ensure all services use same network or no custom networks (default is shared).

Issue 3: Environment variables not working

Debug:

```bash
# Check what variables container sees
docker-compose exec api env | grep DB
```

Common mistakes:

```yaml
# ❌ Wrong: Quotes around variable
environment:
  DB_HOST: "${DB_HOST}"

# ✅ Correct: No quotes
environment:
  DB_HOST: ${DB_HOST}

# ✅ Also correct: Direct value
environment:
  DB_HOST: postgres
```

Issue 4: Volume data not persisting

Check volumes:

```bash
docker volume ls
docker volume inspect <project>_postgres-data
```

Ensure volume is mounted:

```yaml
services:
  postgres:
    volumes:
      - postgres-data:/var/lib/postgresql/data  # Must match

volumes:
  postgres-data:  # Must be declared
```

✅ End of Day 4 Checklist

- [ ] **docker-compose.yml created** - Single file with all services
- [ ] **Multi-service orchestration** - API, database, cache working together
- [ ] **Service dependencies** - Using depends_on with conditions
- [ ] **Health checks** - All services have proper health checks
- [ ] **Compose commands** - Comfortable with up, down, logs, ps, exec

---

🚀 Essential Commands Quick Reference

```bash
```

```
# Start/Stop
docker-compose up -d              # Start all services (detached)
docker-compose down               # Stop and remove containers
docker-compose down -v            # Also remove volumes
docker-compose restart            # Restart all services

# Build
docker-compose build              # Build all services
docker-compose build --no-cache   # Build without cache
docker-compose up --build         # Build and start

# Logs
docker-compose logs               # All logs
docker-compose logs -f            # Follow logs
docker-compose logs api           # Specific service
docker-compose logs --tail=50     # Last 50 lines

# Status
docker-compose ps                 # List containers
docker-compose top                # Show processes

# Execute
docker-compose exec api sh        # Shell in container
docker-compose run api go test    # One-off command

# Validation
docker-compose config             # Check syntax
docker-compose config --services  # List services
```

💡 Key Takeaways

1. **Compose simplifies orchestration**
   - One file replaces dozens of commands
   - Version controlled configuration
   - Easy to share with team
2. **Health checks are critical**
   - Don't rely on depends_on alone
   - Use service_healthy condition
   - Set appropriate start_period
3. **Environment variables for flexibility**

- Use .env for secrets
- Provide defaults with :-
- Different configs for dev/prod

4. **Service communication is automatic**
   - Use service names as hostnames
   - No need to know IPs
   - Automatic DNS resolution
5. **Compose is production-ready**
   - Add resource limits
   - Configure restart policies
   - Implement health checks
   - Set up logging

🔗 Optional Reading

Docker Compose:

- Compose File Reference
- Compose CLI Reference
- Networking in Compose

Best Practices:

- Compose Production Guide
- Health Checks
- Environment Variables

Examples:

- Awesome Compose - Sample applications

🔮 Preview: Tomorrow (Day 5)

Full Stack Setup: API + Postgres + Redis + Frontend

You'll build a complete application with:

- React frontend

- Go API backend
- PostgreSQL database
- Redis cache
- Nginx reverse proxy
- All orchestrated with Compose

Preview:

```yaml
services:
  frontend:
    build: ./frontend
    depends_on:
      - api

  api:
    build: ./api
    depends_on:
      - postgres
      - redis

  nginx:
    image: nginx
    ports:
      - "80:80"
    depends_on:
      - frontend
      - api
```

Great work mastering Docker Compose! 🐬